

Code Explanation

Code Explanation: AWS Glue PySpark Test Suite This is a **comprehensive test suite** for a Customer Order Pipeline built with AWS Glue and PySpark. It validates data transformations, quality rules, and Slowly Changing Dimension (SCD) logic.---

Step 1: Imports and Dependencies

```
import sys
from datetime import datetime, date
from decimal import Decimal
from unittest.mock import patch, MagicMock
import pytest
from pyspark.sql import SparkSession
from pyspark.sql.types import ...
```

What happens:- Imports Python standard libraries for dates, decimals, and mocking- Imports `pytest` testing framework- Imports PySpark components for DataFrame operations and schema definitions- Imports the main job modules to be tested (DataReader, DataCleaner, etc.)---

Step 2: Spark Session Fixture

```
@pytest.fixture(scope="session")
def spark():
    spark = SparkSession.builder \
        .appName("test-customer-order-pipeline") \
        .master("local[2]") \
        .config("spark.sql.shuffle.partitions", "2") \
        .getOrCreate()
    yield spark
    # Cleanup logic
```

What happens:- Creates a **session-scoped** Spark instance (shared across all tests)- Configures local mode with 2 cores for parallel testing- Reduces shuffle partitions to 2 for faster test execution- Uses `yield` to provide the session to tests, then cleans up afterward- Avoids stopping Spark if it's managed by AWS Glue runtime---

Step 3: Sample Data Fixtures

3a. Clean Customer Data

```
@pytest.fixture
def sample_customer_data(spark):
    data = [ ("C001", "John Doe", "john@example.com", "North"), # ... more records ]
    schema = StructType(...)
    return spark.createDataFrame(data, schema)
```

What happens:- Creates a **valid customer dataset** with 5 records- Defines schema with non-nullable fields (StringType)- Returns a PySpark DataFrame for testing

3b. Customer Data with Nulls

```
@pytest.fixture
def sample_customer_data_with_nulls(spark):
    data = [ ("C002", None, "jane@example.com", "South"), # Null name
              ("C003", "Bob Johnson", "Null", "East"), # String "Null"
              # ... ]
```

What happens:- Creates data with **intentional null values** and string "Null"- Used to test data quality validation logic- Schema allows nullable fields (True parameter)

3c. Customer Data with Duplicates

```
@pytest.fixture
def sample_customer_data_with_duplicates(spark):
    data = [ ("C001", "John Doe", "john@example.com", "North"),
              ("C001", "John Doe", "john@example.com", "North"), # Duplicate ]
```

****What happens:****- Creates data with ****duplicate customer records****- Tests deduplication logic in the pipeline

****3d. Order Data****

```
@pytest.fixturedef sample_order_data(spark): data = [ ("0001", "Laptop",  
Decimal("999.99"), 2, date(2024, 1, 15), "C001"), ] schema = StructType([  
StructField("orderid", StringType(), False), StructField("priceperunit",  
DecimalType(10, 2), False), # ... ])
```

****What happens:****- Creates order data with ****proper data types**** (Decimal for prices, Date for dates)- Links orders to customers via `custid` foreign key- Used to test join operations and aggregations---

****Step 4: Job Parameter Tests****

```
def test_get_job_parameters(): params = get_job_parameters() assert params is not  
None assert "customer_source_path" in params assert  
params["customer_source_path"].startswith("s3://")
```

****What happens:****- Tests that job parameters are correctly loaded- Validates required keys exist (source paths, target paths, catalog database)- Ensures all paths use S3 protocol (not local file paths)---

****Step 5: Spark Context Initialization Tests****

```
@patch('main.job.Job')@patch('main.job.GlueContext')@patch('main.job.SparkContext'  
)@patch('main.job.getResolvedOptions')def  
test_initialize_spark_contexts(mock_resolved_options, ...):  
mock_resolved_options.return_value = {'JOB_NAME': 'test-job'} # ... setup mocks  
spark, glueContext, job, args = initialize_spark_contexts()
```

****What happens:****- Uses ****mocking**** to simulate AWS Glue environment without actual AWS resources- Patches Glue-specific classes (GlueContext, Job)- Verifies that initialization returns all required contexts- Checks that job arguments are correctly parsed---

****Step 6: S3 Access Validation Tests****

```
def test_validate_s3_access(spark): assert validate_s3_access(spark,  
"s3://bucket/path/") == True assert validate_s3_access(spark,  
"file:///local/path") == False assert validate_s3_access(spark, None) == False
```

****What happens:****- Tests S3 path validation logic- Ensures only valid S3 URIs are accepted- Rejects local file paths, empty strings, and None values---

****Step 7: DataReader Tests****

****7a. Customer Data Reading****

```
def test_data_reader_customer(spark, sample_customer_data): reader =  
DataReader(spark) # Create data with CamelCase columns (matches actual schema)  
schema = StructType([ StructField("CustId", StringType(), False), # ... ]) with  
patch.object(DataReader, '_read_csv', return_value=sample_data_camelcase): df =  
reader.read_customer_data("s3://test-bucket/customer/")
```

****What happens:****- Tests the DataReader class for customer data- ****Mocks S3 read operation**** to avoid actual AWS calls- Verifies column names match expected CamelCase format- Validates schema

types (all StringType for customer data)- Checks record count matches expected value (5 records)

****7b. Order Data Reading****

```
def test_data_reader_order(spark, sample_order_data): # Create raw data as strings
(before type casting) raw_data = [ ("0001", "Laptop", "999.99", "2", "2024-01-15",
"C001"), ] with patch.object(DataReader, '_read_csv', return_value=raw_df): df =
reader.read_order_data("s3://test-bucket/order/") # Verify type casting assert
df.schema["PricePerUnit"].dataType == DecimalType(10, 2) assert
df.schema["Qty"].dataType == IntegerType()
```

****What happens:****- Tests order data reading with ****type conversion****- Starts with raw string data (simulating CSV input)- Verifies DataReader correctly casts: - Prices to Decimal(10,2) - Quantities to Integer - Dates to DateType- Ensures data transformations happen during read---

****Step 8: DataCleaner Tests****

```
def test_data_cleaner_remove_nulls(spark, sample_customer_data_with_nulls):
df_clean = DataCleaner.remove_nulls(sample_customer_data_with_nulls) # Should
remove 3 records with nulls assert df_clean.count() == 2 # Verify no nulls remain
for row in df_clean.collect(): assert row
```

****What happens:****- Tests null value removal logic- Input: 5 records with 3 containing nulls- Expected output: 2 clean records- Validates that ****all remaining records are complete**** (no null values)- Tests data quality enforcement---

****Key Testing Patterns Used****

1. ****Fixtures****: Reusable test data shared across multiple tests2. ****Mocking****: Simulates AWS services without actual cloud resources3. ****Assertions****: Validates expected behavior (counts, schemas, data types)4. ****Isolation****: Each test runs independently with fresh data5. ****Coverage****: Tests happy paths, edge cases (nulls, duplicates), and error conditions---

****What This Test Suite Validates****

■ Job parameter loading from Glue environment ■ Spark/Glue context initialization ■ S3 path validation ■ CSV data reading with schema enforcement ■ Type casting (strings → decimals, dates, integers) ■ Null value detection and removal ■ Duplicate record handling ■ Data quality rules enforcement This ensures the production pipeline will handle real-world data issues correctly before deployment.